

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

Лабораторная работа №8

Аннотации. Stream API

по дисциплине

«Информационные технологии и программирование»

Выполнил: студент гр. БВТ2403

Кузнецов И.Д.

Руководитель:

Москва, 2025 г.

Цель работы: изучить применение аннотаций и Stream API в Java, освоить функциональные возможности для обработки коллекций данных, а также реализовать многопоточную обработку данных с использованием библиотеки `java.util.concurrent`.

Ход работы:

Аннотации в Java — это метаданные, которые могут быть добавлены к классам, методам, полям и другим элементам программы. Они используются для анализа кода, автоматизации компиляции и управления поведением программы во время выполнения. Аннотации могут иметь различные области видимости и политики хранения (`RetentionPolicy`).

Stream API позволяет работать с коллекциями данных в функциональном стиле. Основные компоненты:

- Источник данных — коллекции, массивы, генераторы и т. д.;
- Промежуточные операции — фильтрация (`filter`), преобразование (`map`), сортировка (`sorted`), удаление дубликатов (`distinct`) и др.;
- Терминальные операции — подсчёт (`count`), сбор в коллекцию (`collect`), итерация (`forEach`), свёртка (`reduce`).

Задание.

- 1) Создание аннотации `@DataProcessor` для пометки методов обработки данных. Аннотация предназначена для указания методов, которые будут использоваться для обработки информации.

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface DataProcessor {
}
```

- 2) Разработка класса `DataManager`, выполняющего многопоточную обработку данных:

- `registerDataProcessor(Object processor)` — регистрирует объект-обработчик;
- `loadData(String source)` — загружает данные из исходного источника;
- `processData()` — применяет методы с аннотацией `@DataProcessor` к данным с использованием Stream API и многопоточности;
- `saveData(String destination)` — сохраняет обработанные данные в новый источник.

```

import java.io.*;
import java.lang.reflect.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;

public class DataManager {
    private List<String> data = new ArrayList<>();
    private final List<Object> processors = new ArrayList<>();
    private final ExecutorService executor = Executors.newFixedThreadPool(4);

    public void registerDataProcessor(Object processor) {
        processors.add(processor);
    }

    public void loadData(String source) throws IOException {
        try (BufferedReader reader = new BufferedReader(new FileReader(source))) {
            data = reader.lines().collect(Collectors.toList());
        }
    }

    public void processData() throws Exception {
        for (Object processor : processors) {
            Method[] methods = processor.getClass().getDeclaredMethods();
            for (Method method : methods) {
                if (method.isAnnotationPresent(DataProcessor.class)) {
                    @SuppressWarnings("unchecked")
                    Future<List<String>> future = executor.submit(() ->
                        (List<String>) method.invoke(processor, data)
                    );
                    data = future.get();
                }
            }
        }
        executor.shutdown();
    }

    public void saveData(String destination) throws IOException {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(destination))) {
            for (String line : data) {
                writer.write(line);
                writer.newLine();
            }
        }
    }
}

```

3) Создание обработчиков данных, помеченных аннотацией @DataProcessor:

- Фильтрация данных по условию FilterProcessor
- Преобразование элементов UpperCaseProcessor
- Агрегация данных AggregateProcessor
- Main

```

import java.util.List;
import java.util.stream.Collectors;

public class FilterProcessor {
    @DataProcessor
    public List<String> filterData(List<String> input) {
        return input.stream()
            .filter(s -> s.startsWith("A"))
            .collect(Collectors.toList());
    }
}

import java.util.List;
import java.util.stream.Collectors;

public class UpperCaseProcessor {
    @DataProcessor
    public List<String> transformData(List<String> input) {
        return input.parallelStream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());
    }
}

import java.util.*;

public class AggregateProcessor {
    @DataProcessor
    public List<String> aggregateData(List<String> input) {
        long count = input.size();
        double avgLength = input.stream()
            .mapToInt(String::length)
            .average()
            .orElse(0.0);

        List<String> result = new ArrayList<>(input);
        result.add("");
        result.add("Всего строк: " + count);
        result.add("Средняя длина: " + String.format("%.1f", avgLength));
        return result;
    }
}

```

```
public class Main {  
    public static void main(String[] args) {  
        DataManager manager = new DataManager();  
  
        manager.registerDataProcessor(new FilterProcessor());  
        manager.registerDataProcessor(new UpperCaseProcessor());  
        manager.registerDataProcessor(new AggregateProcessor());  
  
        try {  
            manager.loadData("input.txt");  
  
            manager.processData();  
  
            manager.saveData("output.txt");  
  
            System.out.println("Данные успешно обработаны!");  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

input.txt:

Apple
Banana
Orange
Melon
Kivi

output.txt:

APPLE
MELON
BANAN

Всего строк: 3

Средняя длина: 6,3

Вывод: лабораторная работа позволила изучить возможности аннотаций и Stream API в Java, а также освоить многопоточную обработку данных. Stream API обеспечивает удобный функциональный подход к обработке коллекций, а аннотации позволяют структурировать код и маркировать методы для автоматической обработки. Применение многопоточности и потокобезопасных структур данных повышает производительность и надёжность приложений.

<https://github.com/Xaminame/ITIP.git>